STOCHASTIC MODELING

MODULE 5 | LESSON 3

HIDDEN MARKOV MODELS (HMM)

Reading Time	80 minutes
Prior Knowledge	Linear Algebra, Maximum likelihood estimator, Monte Carlo

|**Keywords** |Markov Process, Markov chains, Transition Matrix

---

We have already seen the power of Markov chains in asset valuation taking into account changes in the 'state' of the world such as a change in rating. However, this process required knowledge of transition matrix, which is not an easy information to have. Here, we introduce **Hidden Markov Models (HMM)** as a tool that helps in this endeavour.

In [1]:
```
from IPython.display import VimeoVideo

# Bigger video
VimeoVideo("1121457333", h="3298dbabb7", width=700, height=450)
```

Out[1]:

1. Hidden Markov Models (HMM)

A Hidden Markov Model (HMM) is a Markov process that is split into two components: an observable component and an unobservable or "hidden" component that follows a Markov process. HMMs naturally describe setups where a stochastic system is observed through noisy measurements, for instance stock prices, that are affected by an unobserved economic factor. This type of modelization is of broad application in many fields, from speech recognition to thermodynamics. Let's here focus on one financial application of HMMs.

A basic HMM contains the following components:

- A set of N states $\mathcal{S}=\{s_1,...,s_N\}$
- A transition probability matrix P
- A sequence of T, possibly vector-valued, observations $\mathcal{Y}_T=\{y_1,...,y_T\}$
- A sequence of observation marginal likelihoods $f(y_t|s_t=i)$ for each $i=1,..,N$.
- An initial probability distribution $\pi=\{\pi_1,...,\pi_N\}$.

An important assumption is that the hidden Markov process is independent of past observations $\mathcal{Y}_{t-1}$, i.e., $\mathbb{P}\{s_t=j|s_t=i,\mathcal{Y}_{t-1}\}=\mathbb{P}\{s_t=j|s_t=i\}=p_{ij}$.

Conceptually, you could link this to the efficient market hypothesis, whereby current asset prices reflect all (past) available information.

This type of setup is also labeled in some instances as a Markov regime-switching model, where the observable output y_t features a marginal distribution whose parameters change with the realization of an unobservable state.

Let's see this with an example of Hidden Markov Process.

Suppose that $$ y_t=\mu_t+\varepsilon_t $$
where $\mu_t = \mu_j$ if $s_t=j$, $\mathcal{S}=\{1,...N\}$, S_t is Markovian, and ε_t is i.i.d. $N(0,\sigma^2)$.

We are interested in making inference about the probability of being in each state s_i at each date t, as well as estimating the model parameters of the transition matrix P and the vector $(\mu_1,...,\mu_N,\sigma)$. We collect all the parameters in a vector θ and assume for now that we know its value with certainty.

We can build the log-likelihood function of the process:

$$ \begin{align} \mathcal{L}(\theta) = \sum_{t=1}^T log\ f(y_t|\mathcal{Y}_{t-1};\theta) \end{align} $$

where $$ \begin{align} f(y_t|\mathcal{Y}_{t-1};\theta) = \sum_{i=1}^N \mathbb{P}(s_t=i|\mathcal{Y}_{t-1};\theta) \times \phi_{t}(i) \end{align} $$

with $\phi_{t}(i)=\phi\left(\frac{y_t-\mu_i}{\sigma}\right)$. $\phi(.)$ denotes the standard normal probability density function.

The main problem arises from the evaluation of $\mathbb{P}(s_t=i|\mathcal{Y}_{t-1};\theta)$, which would usually require an overwhelming amount of computations for each potential path $\mathcal{Y}_{t-1}$ and for each period t. This rules out estimating directly θ by Maximum Likelihood.

2. Hamilton filter

One potential approach to reach a solution is to denote $\xi_{t|t}(j)=\mathbb{P}(s_t=j|\mathcal{Y}_{t};\theta)$. Then, to estimate the model, we can rely on a recursive algorithm that sets the optimal forecasts $\xi_{t+1|t}(j)=\mathbb{P}(s_{t+1}=j|\mathcal{Y}_{t};\theta)$:

$$ \xi_{t|t}(i) = \frac{\xi_{t|t-1}(i)\phi_{t}(i)}{f(y_{t}|\mathcal{Y}_{t-1};\theta)} $$
$$ \xi_{t+1|t}(j) = \sum_{i=1}^N p_{ij} \xi_{t|t}(i) $$

This procedure is called the **Hamilton filter**, developed by Hamilton (1990). To initialize the recursion, we can set $\xi_{1|0}(i)=1/N$ or, if we have some guess for the initial distribution, $\xi_{1|0}(i)=\pi_i$. Another alternative is to include it in the vector of parameters to estimate, under the usual constraints for a probability distribution.

Then, to estimate the model, we would set initial parameters θ^0 that would allow us to recover each $\xi_{t+1|t}(j)$ and evaluate the log-likelihood function, then iterate with new guess θ^1 and so on until convergence.

Notice that we can make forecasts on the observable process y_{t+1} by exploiting the expressions for $\xi_{t+1|t}(j)$ once we have estimated the model parameters.

3. Hamilton filter in practice

Let's bring to practice this estimation by simulating a time series governed by a Hidden Markov process. We first build a simulated time series and then estimate the model by maximum likelihood, applying the Hamilton filter within the evaluation step.

The idea here is to see how the Hamilton filter allows us to recover otherwise unknown probabilities of different states. To start with, let's go over a tractable example.

3.1. A tractable example

Let's first see how the **Hamilton filter** works by doing an example we can replicate 'by hand'. To start with, we need to define the ingredients of the Markov model.

Ingredients of a simple HMM

Suppose we have a time series with just 3 observations ($T=3$), such that:

$$ Y = (y_1, y_2, y_3) = (-0.85, 0.4, -0.2) $$

We assume that this time series is generated by a Hidden Markov process such that:
$$ y_t = \epsilon_t $$

where the distribution of ϵ_t is normal with mean μ_i and variance σ^2_i. There are 2 states of the world (e.g., recession and expansion) such that:
$$ \boldsymbol{\mu} = (-1, 1) \ \ \text{ and } \ \ \boldsymbol{\sigma} = (0.8, 0.8) $$

Moreover, the matrix of transition probabilities, P, is given by:

$$ \mathbf{P} = \begin{pmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{pmatrix} $$

Lastly, the initial probability distribution for each state, π, is:

$$ \boldsymbol{\pi} = (0.2, 0.8) $$

Remember that, for each observation y_t, the likelihood of observing it in state s is given by:

$$ f(y_t | S_t = s) = \frac{1}{\sqrt{2\pi\sigma_s^2}} \exp\left(-\frac{(y_t - \mu_s)^2}{2\sigma_s^2}\right) $$

And, thus, the total log-likelihood is given by::

$$ \mathcal{L}(\boldsymbol{\theta}; Y) = \log f(y_3|y_2, y_1; \boldsymbol{\theta}) + \log f(y_2|y_1; \boldsymbol{\theta}) + \log f(y_1; \boldsymbol{\theta}) $$

where:

$$ f(y_1; \boldsymbol{\theta}) = P(s_1 = 1; \boldsymbol{\theta}) \times \frac{1}{\sigma_1} \phi\left(\frac{y_1 - \mu_1}{\sigma_1}\right) + P(s_1 = 2; \boldsymbol{\theta}) \times \frac{1}{\sigma_2} \phi\left(\frac{y_2 - \mu_2}{\sigma_2}\right) $$
$$ f(y_2|y_1; \boldsymbol{\theta}) = P(s_2 = 1|y_1; \boldsymbol{\theta}) \times \frac{1}{\sigma_1} \phi\left(\frac{y_2 - \mu_1}{\sigma_1}\right) + P(s_2 = 2|y_1; \boldsymbol{\theta}) \times \frac{1}{\sigma_2} \phi\left(\frac{y_2 - \mu_2}{\sigma_2}\right) $$
$$ f(y_3|y_2, y_1; \boldsymbol{\theta}) = P(s_3 = 1|y_2, y_1; \boldsymbol{\theta}) \times \frac{1}{\sigma_1} \phi\left(\frac{y_3 - \mu_1}{\sigma_1}\right) + P(s_3 = 2|y_2, y_1; \boldsymbol{\theta}) \times \frac{1}{\sigma_2} \phi\left(\frac{y_3 - \mu_2}{\sigma_2}\right) $$

We can simplify these expressions given that:

$$ \frac{1}{\sigma_1} \phi\left(\frac{y_1 - \mu_1}{\sigma_1}\right) = \frac{1}{0.8} \phi\left(\frac{(-0.85 - (-1))}{0.8}\right) \approx 0.49 $$

Analogously:

$$ \frac{1}{\sigma_2} \phi\left(\frac{y_1 - \mu_2}{\sigma_2}\right) = 0.034 $$

$$ \frac{1}{\sigma_1} \phi\left(\frac{y_2 - \mu_1}{\sigma_1}\right) = 0.108 $$

$$ \frac{1}{\sigma_2} \phi\left(\frac{y_2 - \mu_2}{\sigma_2}\right) = 0.376 $$

$$ \frac{1}{\sigma_1} \phi\left(\frac{y_3 - \mu_1}{\sigma_1}\right) = 0.302 $$

$$ \frac{1}{\sigma_2} \phi\left(\frac{y_3 - \mu_2}{\sigma_2}\right) = 0.162 $$

which simplies the different components of the log-likelihood to:

$$ f(y_1; \boldsymbol{\theta}) = P(s_1 = 1; \boldsymbol{\theta}) \times 0.49 + P(s_1 = 2; \boldsymbol{\theta}) \times 0.034 $$

$$ f(y_2|y_1; \boldsymbol{\theta}) = P(s_2 = 1|y_1; \boldsymbol{\theta}) \times 0.108 + P(s_2 = 2|y_1; \boldsymbol{\theta}) \times 0.376 $$

$$ f(y_3|y_2, y_1; \boldsymbol{\theta}) = P(s_3 = 1|y_2, y_1; \boldsymbol{\theta}) \times 0.302 + P(s_3 = 2|y_2, y_1; \boldsymbol{\theta}) \times 0.162 $$

Now its time to estimate the state probabilities, $\xi_{t+1|t}(s)$, using Hamilton's filter. Starting in $t=1$ this implies:

Step 1: Initialize with given parameters (t=1)

$$ P(s_1 = s; \boldsymbol{\theta}) = \pi_s = \xi_{1|0}(s) $$

Step 2: Compute marginal likelihood

$$ f(y_1; \boldsymbol{\theta}) = \pi_1 \times 0.49 + \pi_2 \times 0.034 = 0.2 \times 0.49 + 0.8 \times 0.034 = 0.126 $$

Step 3: Filtering (applying Bayes theorem)

$$ P(s_1 = s|y_1; \boldsymbol{\theta}) = \frac{P(s_1 = s; \boldsymbol{\theta}) \times P(y_1|s_1 = i; \boldsymbol{\theta})}{P(y_1; \boldsymbol{\theta})} = \frac{\pi_s \times \phi\left(\frac{y_1-\mu_s}{\sigma_s}\right)/\sigma_i}{f(y_1; \boldsymbol{\theta})} = \xi_{1|1}^{(s)} $$

As a side note, remember Bayes Theorem:

$$ P(A|B) = \frac{P(B|A)P(A)}{P(B)} $$

Step 4: Predict next period

$$ P(s_2 = s|y_1; \boldsymbol{\theta}) = p_{1s} \xi_{1|1}(1) + p_{2s} \xi_{1|1}(2) = \xi_{2|1}(s) $$

Following this process, we get:

$$ \xi_{1|1}(1) = 0.781; \quad \xi_{1|1}(2) = 0.219; \quad \xi_{2|1}(1) = 0.668; \quad \xi_{2|1}(2) = 0.332 $$

Once we have this for $t=1$, we can analogously do it for $t=2$, repeating the same process:

$$ f(y_2|y_1; \boldsymbol{\theta}) = \xi_{2|1}(s=1) \times 0.108 + \xi_{2|1}(s=2) \times 0.376 = 0.197 $$

$$ P(s_2 = s|y_2, y_1; \boldsymbol{\theta}) = \frac{P(s_2 = s|y_1; \boldsymbol{\theta}) \times P(y_2|s_2 = s, y_1; \boldsymbol{\theta})}{P(y_2|y_1; \boldsymbol{\theta})} = \frac{\xi_{2|1}(s) \times \phi\left(\frac{y_2-\mu_s}{\sigma_s}\right)/\sigma_s}{f(y_2|y_1; \boldsymbol{\theta})} = \xi_{2|2}(s) $$

$$ P(s_3 = s|y_2, y_1; \boldsymbol{\theta}) = p_{1s} \xi_{2|2}(s=1) + p_{2s} \xi_{2|2}(s=2) = \xi_{3|2}(s) $$

which yields:

$$ \xi_{2|2}(1) = 0.366; \quad \xi_{2|2}(2) = 0.634; \quad \xi_{3|2}(1) = 0.42; \quad \xi_{3|2}(2) = 0.58 $$

Finally, for period $t=3$ we repeat again:

$$ f(y_3|y_2, y_1; \boldsymbol{\theta}) = \xi_{3|2}(1) \times 0.302 + \xi_{3|2}(2) \times 0.162 = 0.221 $$

$$ P(s_3 = i|y_3, y_2, y_1; \boldsymbol{\theta}) = p_{1s} \xi_{3|2}(1) + p_{2s} \xi_{3|2}(2) = \xi_{3|3}(s) $$

$$ \xi_{3|3}(1) = 0.575; \quad \xi_{3|3}(2) = 0.425 $$

This would produce a log-likelihood given by:

$$ \mathcal{L}(\boldsymbol{\theta}; Y) = \log f(y_1; \boldsymbol{\theta}) + \log f(y_2|y_1; \boldsymbol{\theta}) + \log f(y_3|y_2, y_1; \boldsymbol{\theta}) = \log(0.126) + \log(0.197) + \log(0.221) = -5.202 $$

So far, during this tedious process, we have implemented the Hamilton procedure just to evaluate the 'forward pass' of the model. That is, given some initial parameters, we can use Hamilton's procedure to estimate the probabilities of being at each state and evaluate the overall log-likelihood.

The next natural step would be to recursively estimate model parameters seeking maximum likelihood. Let's implement this in coding.

3.2. Hamilton filter example in python

Next, we are going to verify that using a number of simple functions defined in python we can yield the same results as in the previous hand-written example.

For starters, let's import some useful libraries:

In [2]:
```
import numpy as np
from scipy.optimize import minimize
from scipy.stats import norm
import matplotlib.pyplot as plt
```

And let's define some functions:

In [3]:
```
def emission_prob(y, mu, sigma):
    """Calculate emission probability: f(y_t | s_t = i)"""
    return norm.pdf(y, loc=mu, scale=sigma)


def hamilton_filter(y, mu, sigma, P, pi):
    """
    Hamilton Filter implementation

    Parameters:
    y: observations (T,)
    mu: values for each state (n_states,)
    sigma: std devs for each state (n_states,)
    P: transition matrix (n_states, n_states)
    pi: initial probabilities (n_states,)

    Returns:
    filtered_probs: filtered probabilities (T, n_states)
    predicted_probs: predicted probabilities (T, n_states)
    log_likelihood: total log likelihood
    marginal_likelihoods: marginal likelihoods at each step
    """
    T = len(y)
    n_states = len(mu)

    # Storage
    filtered_probs = np.zeros((T, n_states))
    predicted_probs = np.zeros((T, n_states))
    marginal_likelihoods = np.zeros(T)

    # Initialize with prior
    predicted_probs[0] = pi

    for t in range(T):
        # Step 1: Calculate emission probabilities for all states (f( ))
        emissions = np.array([emission_prob(y[t], mu[i], sigma[i])
                              for i in range(n_states)])

        # Step 2: Calculate marginal likelihood
        marginal_likelihoods[t] = np.sum(predicted_probs[t] * emissions)

        # Step 3: Update (filter) - Bayes rule
        filtered_probs[t] = (predicted_probs[t] * emissions) / marginal_likelihoo

        # Step 4: Predict next period (if not last observation)
        if t < T - 1:
```

```
            predicted_probs[t + 1] = P.T @ filtered_probs[t]

    # Calculate log likelihood
    log_likelihood = np.sum(np.log(marginal_likelihoods))

    return filtered_probs, predicted_probs, log_likelihood, marginal_likelihoods
```

With these we should be able to replicate the hand-written example, given the initial parameters:

In [4]:
```
# Initial data
y = np.array([-0.85, 0.4, -0.2])
mu = np.array([-1.0, 1.0])
sigma = np.array([0.8, 0.8])
P = np.array([[0.8, 0.2],
              [0.2, 0.8]])
pi = np.array([0.2, 0.8])
```

In [5]:
```
# Run Hamilton filter
filtered_probs, predicted_probs, log_likelihood, marginals = hamilton_filter(y,
```

Let's check the results and whether they align with previous ones:

In [6]:
```
print(f"Total log-likelihood: {log_likelihood:.3f}")
print("Predicted probabilities (before observing y_t):")
for t in range(len(y)):
    if t == 0:
        print(f"  ξ_{t+1}|{t} = {predicted_probs[t]} (initial π)")
    else:
        print(f"  ξ_{t+1}|{t} = [{predicted_probs[t,0]:.3f}, {predicted_probs[t,
```

```
Total log-likelihood: -5.210
Predicted probabilities (before observing y_t):
  ξ_1|0 = [0.2 0.8] (initial π)
  ξ_2|1 = [0.668, 0.332]
  ξ_3|2 = [0.420, 0.580]
```

As you can see, we obtain essentially the same numbers (except for some rounding error) in a much more efficient way. This allows us to recursively estimate model parameters (i.e., θ vector) by maximizing log-likelihood (or, equivalently, minimizing negative-log-likelihood).

3.3. Estimating parameters by maximizing log-likelihood

Now we can setup functions to obtain the log-likelihood and perform estimation of model parameters. The iterative method to estimate these parameters is usually known as **Expectation-Maximization (EM) algorithm**:

In [7]:
```
def negative_log_likelihood(params, y):
    """
    Negative log likelihood function for optimization
    We MINIMIZE this function to MAXIMIZE the likelihood

    Parameters format: [mu1, mu2, sigma1, sigma2, p11, p22, pi1]
    """
```

```
    mu1, mu2, sigma1, sigma2, p11, p22, pi1 = params

    # Basic constraints to avoid crazy results
    if sigma1 <= 0 or sigma2 <= 0:
        return 1e10
    if sigma1 >= 1 or sigma2 >= 1:
        return 1e10
    if not (0 <= p11 <= 1) or not (0 <= p22 <= 1):
        return 1e10
    if not (0 <= pi1 <= 1):
        return 1e10

    # Build parameter arrays
    mu = np.array([mu1, mu2])
    sigma = np.array([sigma1, sigma2])
    P = np.array([[p11, 1-p11],
                  [1-p22, p22]])
    pi = np.array([pi1, 1-pi1])

    # Run Hamilton filter to get log-likelihood
    try:
        _, _, log_likelihood, _ = hamilton_filter(y, mu, sigma, P, pi)
        return -log_likelihood
    except:
        return 1e10
```

In [8]:

```
def estimate_mle(y, initial_guess=None):
    """
    Maximum Likelihood Estimation using Hamilton Filter

    We find: θ* = argmax ℒ(θ) = argmin(-ℒ(θ))
    Note that maximizing log-likelihood is equivalent to minimizing negative-log
    """
    if initial_guess is None:
        initial_guess = [-1.0, 1.0, 0.8, 0.8, 0.8, 0.8, 0.2]

    print(f"Initial guess: {initial_guess}")


    # Try optimization
    result = minimize(negative_log_likelihood, initial_guess, args=(y,), method=

    print(f"Optimization result: {result.success}")
    if not result.success:
        print(f"Warning: {result.message}")
        print("Results may be unreliable with only 3 observations!")

    # Extract estimated parameters
    mu1, mu2, sigma1, sigma2, p11, p22, pi1 = result.x

    mu_est = np.array([mu1, mu2])
    sigma_est = np.array([sigma1, sigma2])
    P_est = np.array([[p11, 1-p11], [1-p22, p22]])
    pi_est = np.array([pi1, 1-pi1])

    # Maximum log-likelihood is negative of minimized value
    max_log_likelihood = -result.fun

    return mu_est, sigma_est, P_est, pi_est, max_log_likelihood
```

So now we can estimate parameters from the data, given initial guesses:

In [9]:
```
mu_est, sigma_est, P_est, pi_est, max_log_likelihood = estimate_mle(y)
```

```
Initial guess: [-1.0, 1.0, 0.8, 0.8, 0.8, 0.8, 0.2]
Optimization result: False
Warning: Desired error not necessarily achieved due to precision loss.
Results may be unreliable with only 3 observations!
```

In [10]:
```
print(f"\n- ESTIMATED PARAMETERS -")
print("-" * 25)
print(f"μ₁  {mu_est[0]:.3f}")
print(f"μ₂  {mu_est[1]:.3f}")
print(f"σ₁  {sigma_est[0]:.3f}")
print(f"σ₂  {sigma_est[1]:.3f}")
print(f"π₁  {pi_est[0]:.3f}")
print(f"π₂  {pi_est[1]:.3f}")
```

```
- ESTIMATED PARAMETERS -
-------------------------
μ₁  -0.318
μ₂  0.044
σ₁  0.409
σ₂  0.274
π₁  0.971
π₂  0.029
```

As we have highlighted in the previous code, it's likely that our estimation does not yield the desired results. This is somewhat obvious at first sight, since the amount of data we can work with is really reduced (just 3 observations!). But for now, you know how to implement (and trace back) a Hamilton filter in a Hidden Markov Model.

4. Conclusion

Well done! Implementing HMM in practice clearly presents challenges. We have seen this already with the practical implementation of Hamilton's filter. However, the idea is that you understand the basics behind the implementations. Once we put things into coding, the process is more friendly.

Hamilton filter is the first step towards being able to forecast probabilities of regime switching, which has important implications for stock market forecasting. This is what we are going to see in the next lesson.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.